SAVEETHA SCHOOL OF ENGINEERING

C PROGRAMMING ASSIGNMENT

Comprehensive System Design and Implementation Challenge

Student Name	BAVADARANI S
Register Number	1912260562
Course Code & Name	CSA0209 C PROGRAMMING
Slot	B
Assignment / Task Title	Comprehensive System Design and Implementation Challenge – District Blood Bank Network
GitHub Repository URL	https://github.com/bavadarini28 / CSA0209-1912260562-BAVADARANI.S.git
Submission Date	13 NOVEMBER 2026

TASK A1 — Decompose the Program into Functions (7 Marks)

Task / Problem Statement

Break the blood bank program (register donors, check eligibility, find eligible donors, update stock after every donation and issue, keep data in files) into functions, design their interfaces, and design a function that returns two values.

Aim

To divide the program into small single-purpose functions with clean interfaces so that each can be tested and reused separately.

A1.1 – Identify the Functions (2 marks)

Function	Purpose (one job each)	Calls
main	Reads argc/argv, validates the command, calls the right function, frees memory	all command functions
register_donor	Validates and adds a new donor, then saves	valid_id, group_index, add_donor, save_donors
valid_id	Checks the check digit of a donor ID	digital_root
digital_root / digit_sum	Recursive digital root (check digit)	each other / itself
is_eligible	True if age 18–65 and at least 90 days since last donation	days_from_date
find_eligible (find_donors)	Counts and lists eligible donors of one group; finds the youngest	is_eligible
load_donors / save_donors	Read / write the text donor file	add_donor, fopen/fgets/fprintf
add_donor	Appends to the growable list, doubling with realloc	realloc

Function	Purpose (one job each)	Calls
load_stock / update_units / change_stock	Read the binary stock file / update one record in place	fseek, fread, fwrite
lowest_stock	Returns a pointer to the blood group with least stock	–
group_index	Converts "O+" into 0..7	strcmp
days_from_date / date_from_days	Date text ↔ day number	timegm / gmtime
free_list	Releases the donor list	free

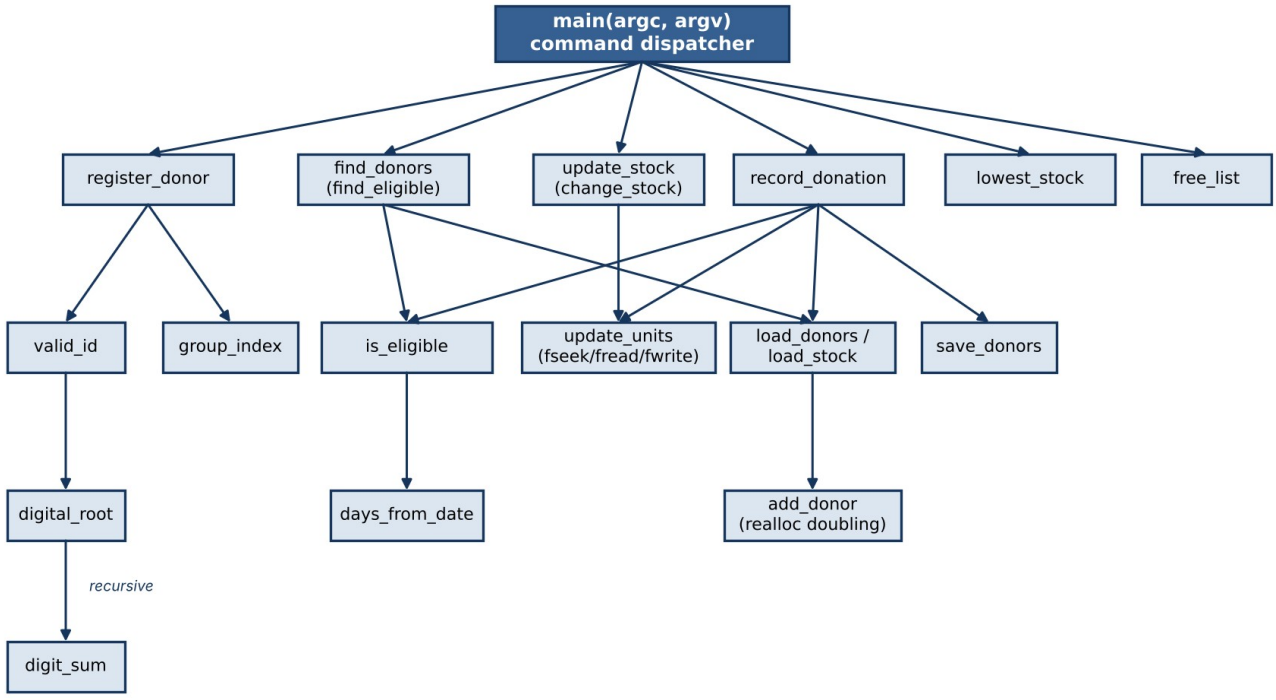


Figure A1 – Structure chart (arrow = "calls").

A1.2 – Design the Interfaces (3 marks)

Rules used for every decision: (1) **small scalars (int, long) are passed by value** – copying is cheap and the caller's variable is protected; (2) **arrays, strings and structures are passed by pointer** – C cannot pass arrays by value, and copying a 72-byte Donor for every call is wasteful; (3) a pointer that is **only read is const** – the compiler then rejects accidental writes and the reader knows the function is harmless; (4) a pointer the function **must change is non-const**.

Prototype	Parameter decisions and justification
int group_index(const char *g)	g by pointer (a string is an array) and const – only compared, never changed.
int digit_sum(long n) int digital_root(long n) int valid_id(int id)	Single numbers, by value – the function works on its own copy, so the caller's ID is never damaged. Return value carries the result.
long days_from_date(const char *s)	s read-only string → const char *. Returns −1 if the text is not a date.
void date_from_days(long d, char out[11])	d by value; out by (non-const) pointer because the function **writes** the text into the caller's buffer.

Prototype	Parameter decisions and justification
int is_eligible(const Donor *d, long today)	d by const pointer: avoids copying 72 bytes and guarantees the record is not modified. today by value.
int add_donor(DonorList *list, const Donor *d)	list by non-const pointer: size, capacity and the data pointer all change. d is only copied from → const.
void free_list(DonorList *l)	Non-const: sets data to NULL and size to 0 after freeing (prevents dangling pointer / double free).
int load_donors(const char *path, DonorList *list)	path const string; list is an **output** parameter so non-const.
int save_donors(const char *path, const DonorList *list)	Saving only reads the list → const DonorList *.
int load_stock(const char *path, Stock *s)	s is the caller's array of 8 records that the function fills → non-const pointer.
int update_units(FILE *fp, int index, int units)	FILE * is a handle that must be shared with the caller (file position changes). index, units by value.
int change_stock(const char *path, int index, int new_units)	Opens, calls update_units, closes. All inputs are read-only values.
void find_eligible(const DonorList *l, const char *group, long today, int *count, int *youngest_id, int print)	List and group are read-only → const. count and youngest_id are **output** pointers (two results). today, print by value.
Stock *lowest_stock(Stock *s, int n)	Non-const Stock * in and out, because the caller will use the returned pointer to **modify** the element. n by value.
int main(int argc, char *argv[])	Standard: argc count by value; argv array of strings supplied by the system.

A1.3 – Return More Than One Value (2 marks)

Alternative 1 – pointer (output) parameters (the version used in the program):

```
ALGORITHM FindEligible(list, group, today, count*, youngestId*)
  INPUT  : list (read-only), group, today
  OUTPUT : *count = number of eligible donors,
           *youngestId = ID of the youngest eligible donor (-1 if none)
BEGIN
  *count <- 0 ; *youngestId <- -1 ; youngestAge <- 1000
  FOR i <- 0 TO list.size - 1 DO
    d <- list.data[i]
    IF d.group = group AND IsEligible(d, today) THEN
      *count <- *count + 1
      IF d.age < youngestAge THEN
        youngestAge <- d.age ; *youngestId <- d.id
      END IF
    END IF
  END FOR
END
```

```
void find_eligible(const DonorList *l, const char *group, long today, int *count, int *youngest_id);
/* call */   int n, id;   find_eligible(&list, "O+", today, &n, &id);
```

Alternative 2 – return a structure:

```
typedef struct { int count; int youngest_id; } EligibleResult;
EligibleResult find_eligible_s(const DonorList *l, const char *group, long today);
/* call */   EligibleResult r = find_eligible_s(&list, "O+", today);   /* r.count, r.youngest_id */
```

Point	Pointer parameters	Return a structure

Call syntax	Caller must declare variables and pass &n, &id	One call, one result variable – reads like a normal function
Cost	Two stores through pointers; nothing copied	8-byte struct is returned in registers by most compilers – equally cheap; a very large struct would be copied
Safety	Caller can pass NULL or forget to read both outputs	Fields are named, no NULL to check
Return value free for errors	Yes – function can return 0/1 status as well	Status must become another field
Extending later (add a 3rd result)	Prototype changes for every caller	Add a field; old calls still compile
Best used when	Many results, large results, or status code needed	A small fixed bundle of related results

Conclusion: for exactly two small related values the structure is cleaner and safer; the pointer version is the classic C idiom, keeps the return value for a status code and avoids copying when results are big. Both run in O(n) time and O(1) extra space.

Alternative Concept and Complexity Analysis – shared global variables

What the solution would be. Declare DonorList donors; Stock stock[8]; int eligible_count; int youngest_id; at file level. Then find_eligible() takes no list argument, reads donors directly and writes eligible_count and youngest_id; add_donor(id, group) modifies donors, and so on.

Aspect	Parameters / pointers	Global variables
Running time	O(n) for find_eligible	Still O(n). Reading a global is as fast as reading through a pointer (a few cycles either way).
Memory	List lives where the caller puts it; stack holds a few pointers	Same data, same size. Globals live for the whole program (static storage), so the memory can never be released early. Saves only a few bytes of parameter passing.
Coupling	Low – a function depends only on what is in its prototype	High (“common coupling”). Every function silently depends on the same hidden variables; a change in one place can break another.
Testing	Build a small list in the test, call, compare	Test must first set up and afterwards reset the global state; tests influence each other and depend on the order they run in.
Reuse	Same function works on any list (e.g. one list per hospital)	Works on exactly one list. A second list or a second thread needs the source changed. Not thread-safe.
Readability	Prototype documents inputs and outputs	Reader must read the whole body to learn what it reads and changes.

Conclusion: the time and memory are essentially unchanged, but the engineering quality is worse. Parameters and pointers are therefore used in this design; the only global in the code is the instrumentation counter calls in B1, which is not part of the blood bank logic.

Result

The program is divided into 13 small functions with a clear call hierarchy. Every parameter is passed by value or by (const) pointer for a stated reason, and two designs for returning two values were compared.